THE BUILD

Six tools that turn a clever prompt into a system

Roles · Structure · Guardrails · Chains · Memory · Agents



LAST ISSUE YOU LEARNED TO WRITE ONE GOOD PROMPT.
THIS ISSUE YOU BUILD A MACHINE OUT OF THEM.

OPEN THE COURSE >

SESSION 2

LESSON 05 — ROLES & PERSONAS

WHO IS IT BEING?

A role sets vocabulary, judgement and defaults — in one line.

- A good role answers three questions: who are you, who are you talking to, and what do you optimise for?
- It changes STYLE and JUDGEMENT. It does NOT add knowledge — "you are a doctor" doesn't tell it your patient's history.
- It grants no real authority and lifts no safety limits.
- The SYSTEM PROMPT is the role set once, governing every conversation — your product's personality and its safety policy.



SET THE REGISTER

WEAK

"Reply to this angry customer." -> a generic corporate apology that promises things you don't even offer.

GOOD

"You are the owner of a small coffee shop replying personally to an upset regular. Warm, no corporate script. You may offer one free drink — nothing more. Under four sentences."

MAKE THE ROLE LOAD-BEARING

- Name who it is, who it serves, and what it optimises for.
- Pair the role with its hard limits — they are strongest together.
- Treat the system prompt as a secret worth protecting, but never put real secrets inside it.

NEXT A ROLE SHAPES THE WORDS. NOW SHAPE THE SHAPE.

LESSON 06 — STRUCTURED OUTPUT

TALK TO SOFTWARE



CUT THE SHAPE

The moment a MACHINE reads the answer, prose becomes a liability.

- Give the exact schema — the keys and their types — plus one example.
- Say "return ONLY the JSON, no prose, no markdown fences".
- Always give a missing-value rule: use null, never invent a field.
- Messy text in, clean data out — this is the engine behind most real prompt products: invoices, reviews, CVs, tickets.

WEAK

"Read this review and tell me the rating and what they liked." -> a paragraph you still have to read and retype somewhere.

GOOD

"Return ONLY JSON: { stars: 1-5, liked: [string], disliked: [string] }." -> { "stars": 3, "liked": ["coffee"], "disliked": ["wifi down"] } — straight into a dashboard.

MAKE IT RELIABLE

- Validate the schema in code and retry on malformed output.
- Many APIs have a strict JSON / schema mode — use it.
- This is the leap from chatbot to SYSTEM: { "priority": "high" } can route a ticket with no human in the middle.

NEXT IT ANSWERS WELL. NOW STOP IT ANSWERING WRONGLY.

THE HONEST MACHINE

The single most valuable line you can write gives it a way out.



HOLD THE LINE

- "If you are not sure, or it isn't in what I gave you, say so or ask. Do not guess." — this one clause separates a demo from a product.
- Prefer positive framing; naming a thing can summon it. If a negative is essential, say what to do INSTEAD.
- Scope limits keep it useful and make it harder to hijack.
- Five-point checklist: escape hatch, scope, the dangerous invention in YOUR domain, a fallback to a human, and grounding.

WEAK

"You're a support bot for our shop. Answer customer questions." -> invents a loyalty programme, wrong holiday hours, writes poems on request.

GOOD

"Answer only about our menu, hours, location and orders, from the info below. If it isn't covered, say 'Let me check.' Decline off-topic. Never invent prices, hours or promotions."

WHAT A CLIENT IS REALLY BUYING

- A bot that admits uncertainty is trustworthy; one that fabricates is a liability.
- Guardrails are most of what makes a workflow sellable.
- Try to break your own assistant before a customer does.

NEXT ONE PROMPT CAN'T DO IT ALL. BREAK IT APART.

ONE JOB PER LINK

A mega-prompt does five things adequately and fails invisibly.

- Three patterns: SEQUENTIAL (refine in stages), ROUTER (classify, then send to a specialist), MAP-REDUCE (same prompt over many pieces, then combine).
- Pass STRUCTURED data between steps, never prose — that's what makes a chain reliable.
- Mix prompts with plain code. Use the model only where judgement is needed; everything else is cheaper and exact in code.
- Log every step's input and output — that log becomes your eval set.



SPLIT THE WORK

WEAK

One prompt: "read this email, classify it, draft a reply in their language, and list the action items." Four jobs, one score, no way to tell which half broke.

GOOD

Four steps: classify -> draft -> translate -> extract actions. Each testable alone. Translations go wrong? You fix ONE prompt and leave the rest.

DEBUG THE LINK, NOT THE ROPE

- Feed a known input to step 1 — is the output correct and well-formed?
- Feed that to step 2. Continue. The first wrong step is your bug.
- This is the leap from prompting to building systems.

NEXT IT STILL ONLY KNOWS WHAT YOU PASTE. OPEN THE VAULT.

GIVE IT YOUR WORLD



OPEN THE VAULT

You can't paste a 300-page manual into every question — so fetch the slice.

- Two jobs bolted together: RETRIEVAL (find the relevant pieces) then GENERATION (answer from them, grounded).
- EMBEDDINGS, plainly: text turned into numbers that capture MEANING, so similar meanings sit close together — search by meaning, not keywords.
- Prepare once: chunk the docs, embed, store. Per question: embed the question, grab the nearest chunks, put them in the prompt.
- Update a document and the answers update — no retraining, ever.

WEAK

"What's the warranty on our espresso machines?" asked of a bare model -> a confident, invented warranty.

GOOD

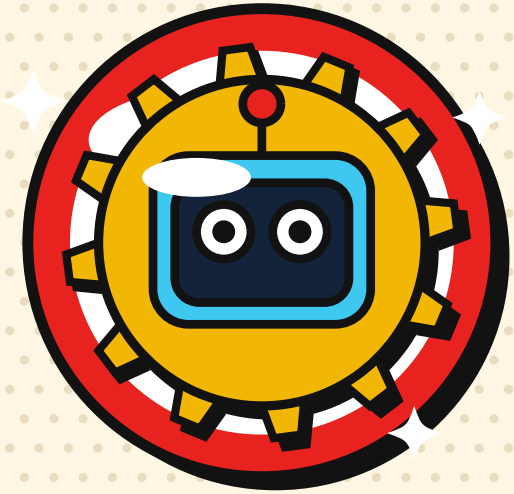
The system pulls the real warranty section from YOUR docs and prompts: "Answer using only the text below; if it doesn't cover it, say you'll check." -> the real answer, or an honest escalation.

ALMOST EVERY FAILURE IS A RETRIEVAL FAILURE

- Wrong chunks fetched = confident wrong answer. The prompt isn't the bug.
- Tune chunk size; retrieve more candidates, then filter.
- Don't reach for RAG when a simple paste will do — it is real overhead.

NEXT IT CAN ANSWER ANYTHING NOW. WHAT IF IT COULD ACT?

FROM TALKING TO DOING



LET IT ACT

A model alone can only produce text. Tools let it act.

- THE LOOP: think -> act (call a tool) -> observe the result -> repeat until done.
- The model doesn't run the tool. It EMITS a structured request; your code runs it and feeds the result back. (Lesson 06, doing real work.)
- Tool descriptions ARE prompts — a vague one gets misused, a precise one gets used correctly.
- Retrieved text and tool results are DATA to consider, never commands to obey — that boundary matters most once it can act.

WEAK

Give it a send_email tool and say "handle customer emails." It can email the wrong person, promise the wrong thing, or be tricked into forwarding your customer list.

GOOD

Read-only tools that DRAFT replies, a human approval step before anything sends, a hard step cap, and email contents treated as data. It drafts; a person clicks send.

THE SAFETY MODEL IS THE HARD PART

• For each tool ask: read-only or does-something? Reversible or not?
• Least privilege — don't hand it tools it doesn't need.
• The less undoable the action (money, deletion, outbound), the more it needs an explicit human confirmation.

NEXT YOU'VE BUILT THE MACHINE. NOW PROVE IT WORKS.

THE WHOLE MACHINE

SIX TOOLS. ONE SYSTEM



ROLE
who it is



STRUCTURE
a shape code can read



GUARDRAILS
what it must never do



CHAINS



MEMORY



AGENTS

A REAL SUPPORT SYSTEM IS A ROUTER IN FRONT, RAG FOR QUESTIONS, AN AGENT FOR ORDERS — EACH WITH ITS OWN ROLE, SCHEMA AND GUARDRAILS.

START SMALL

- Build ONE grounded assistant first.
- Add a chain only when a step clearly needs it.
- Complexity you don't need is cost and risk, not sophistication.

NEXT YOUR ASSIGNMENT IS WAITING

YOUR TURN

ASSIGNMENT — SESSION 2



OPEN THE ASSIGNMENT >

No code required.

1. **Write a load-bearing ROLE**
   who it is, who it serves, what it optimises for — plus its one hard limit.
2. **Force a SCHEMA**
   pick a messy text you handle often; return only JSON, null for anything missing. Test on 5 real inputs.
3. **Fit the five GUARDRAILS**
   escape hatch, scope, the dangerous invention, a human fallback, grounding. Then try to break it.
4. **Split it into a CHAIN**
   take one overloaded prompt, draw the steps, pass JSON between them.
5. **Design the RETRIEVAL**
   sketch how you'd chunk your docs and what one question should fetch.
6. **Draw the AGENT's safety model**
   list the tools; mark read-only vs acting, reversible vs not, and which need human approval.

NEXT — NEXT ISSUE: EVALUATION, SECURITY, COST — AND THE MONEY

TO BE CONTINUED

Issue #3: evaluation · prompt security · cost · the business



- Lesson 05 — Roles & Personas >
- Lesson 06 — Structured Output >
- Lesson 07 — Constraints & Guardrails >
- Lesson 08 — Chaining & Pipelines >
- Lesson 09 — Retrieval (RAG) >
- Lesson 10 — Agents & Tools >
- Session 2 assignment >
- Weak vs Good — paired prompts >